



AIRI



Национальный
исследовательский
Томский
государственный
университет

Отчет по финальному проекту в рамках школы

Летняя школа по искусственному интеллекту Института AIRI

Команда #6:

Benchmarking pre-training methods for
learning from video in reinforcement and
imitation learning

Члены команды:

1. Сергей Давиденко
2. Мария Румянцева
3. Захар Варфоломеев

Ментор:

Александр Никулин
Научный сотрудник AIRI

Лето с AIRI

10 июля 2025 г.

Содержание

1	Введение в задачу	2
1.1	Что такое RL	2
1.2	Online vs Offline RL	2
1.3	Связь RL и VLA	2
1.4	RL в робототехнике	3
1.5	offline RL и данные	3
1.6	Классификация RL подходов	4
1.7	LARO	5
1.8	Методология LARO	6
2	Основная часть	8
2.1	Метагипотеза	8
2.2	Методология	9
2.3	Eval	10
3	Заключение	12

Введение в задачу

Что такое RL

Reinforcement Learning (RL), или обучение с подкреплением, — метод машинного обучения, в котором агент учится принимать решения, взаимодействуя со средой [1]. Основные компоненты RL:

- Агент (Agent): Сущность, которая обучается и принимает решения.
- Среда (Env): Окружение, с которым взаимодействует агент.
- Действие (Action): Шаг, который агент предпринимает в среде.
- Награда (Reward): Обратная связь от среды, оценивающая качество действия.
- Состояние (State): Текущая ситуация в среде.

Цель агента — максимизировать суммарную награду, выбирая оптимальные действия. RL широко применяется в играх, робототехнике, управлении автономными системами и других областях.

Online vs Offline RL

Online RL предполагает непрерывное взаимодействие агента со средой в процессе обучения, что позволяет динамически обновлять политику на основе актуального опыта. В отличие от этого, Offline RL основывается на обучении с фиксированным набором данных, собранных заранее, без возможности дополнительного сбора траекторий в процессе оптимизации [1]. Эти подходы различаются как по методологии сбора данных, так и по применяемым алгоритмам обучения, что обусловлено фундаментальными различиями в постановке задачи.

Связь RL и VLA

Vision-Language-Action (VLA) модели представляют собой новый класс мультимодальных архитектур, объединяющих три ключевых компонента искусственного интеллекта: компьютерное зрение (vision), обработку естественного языка (language) и обучение с подкреплением (action) [2]. Теоретической основой таких моделей служит концепция embodied AI, где агент обучается выполнять физические действия на основе мультимодального восприятия окружающей среды.

В современных VLA-архитектурах RL выполняет двойную функцию:

- Оптимизация политик ($\pi : (V, L) \rightarrow A$), где визуальные (V) и языковые (L) входы преобразуются в пространство действий (A)

- Обучение латентным представлениям через максимизацию функции награды $R(s, a)$.

Типовая схема включает:

- Мультимодальный энкодер (ViT + LLM) для совместного представления визуально-текстовых данных.
- Пространство латентных действий Z с размерностью $d \ll \dim(A)$.
- Адаптивный декодер $z \mapsto a$, обучаемый методом обратного распространения ошибки.

RL в робототехнике

Локомоция – обучение устойчивому передвижению. RL позволяет роботам адаптироваться к сложному рельефу, сохранять баланс и восстанавливаться после сбоев. Например, четвероногие роботы (Boston Dynamics) учатся ходить, бегать и преодолевать препятствия через симуляцию и последующий перенос в реальность. RL успешно обучает роботов (например, Boston Dynamics Spot, ANYmal) устойчиво передвигаться по ровным и умеренно сложным поверхностям [2]. Также современные алгоритмы (PPO, SAC) позволяют роботам восстанавливать баланс после толчков или скольжения, а методы доменной рандомизации (DR) помогают переносить навыки из виртуальной среды в реальную (sim2real).

В то время как задачи локомоции (устойчивое передвижение, балансировка) успешно решаются с помощью RL, задачи манипуляции — точного управления объектами — остаются вызовом. Причина — высокая размерность пространства действий, сложность обратной связи и необходимость точного позиционирования. Для повышения эффективности обучения в манипуляционных задачах часто используют комбинацию RL и имитационного обучения (Learning from Demonstrations) [3], что позволяет ускорить освоение сложных навыков и повысить надёжность работы с хрупкими или нестандартными объектами (например, в коллаборативных манипуляторах UR, xArm и др.).

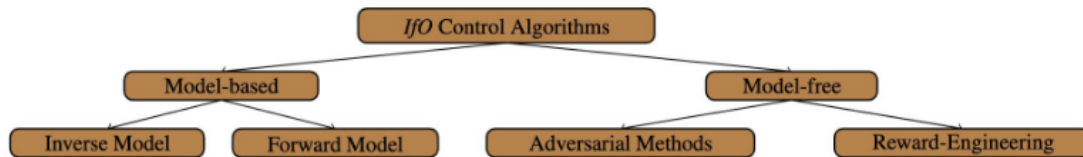
offline RL и данные

Offline RL (или "batch RL") — это обучение с подкреплением без взаимодействия со средой в реальном времени, исключительно на заранее собранных данных.

Особенности Offline RL:

Зависимость от качества данных: Эффективность алгоритмов сильно зависит от объема, разнообразия и репрезентативности датасета.

Проблема распределенного сдвига (distributional shift): Модель может выдавать некорректные действия, если данные не покрывают все возможные состояния и действия.



Требования к данным:

1. Большие объемы: для стабильного обучения требуется значительное количество примеров взаимодействий (например, logs человеческих диалогов для RLHF).
2. Разнообразие сценариев: данные должны включать широкий спектр состояний и действий, чтобы избежать переобучения на узкие паттерны.
3. Аннотации и обратная связь : для задач, подобных RLHF, критически важны метки (например, предпочтения человека между парой ответов).
4. Применение в LLM/VLM: Offline RL используется, когда онлайн-взаимодействие дорого (например, развертывание модели в продакшене) или опасно (риск генерации вредоносного контента) или сложно организуется (роботы). Пример — дообучение ChatGPT на исторических данных диалогов с экспертной разметкой.

Классификация RL подходов

разделяет методы RL на две основные категории: model-based (с использованием модели среды) и model-free (без явной модели), а также включает дополнительные специализированные подходы.

1. Model-based RL. Методы, которые явно обучают или используют модель динамики среды (предсказание следующего состояния и вознаграждения). Это позволяет планировать действия, симулируя будущие состояния, что особенно полезно при ограниченных данных или дорогих экспериментах. Решение такой задачи чрезвычайно актуально для манипуляционной робототехники.
 - (a) Inverse Model - Алгоритмы, предсказывающие действие, необходимое для перехода между двумя состояниями, например, имитационное обучение (обратная динамика).
 - (b) Forward Model — Прямое предсказание следующего состояния и вознаграждения на основе текущего состояния и действия (Dyna-Q, World Models.)
 - (c) Adversarial Methods — Использование состязательных подходов (например, GAN) для улучшения модели среды или политики (Robust RL через adversarial training).

2. Model-free RL. Методы, которые обучают политику или функцию ценности напрямую, без явной модели среды.
 - (a) Reward-Engineering — Специальная настройка функции вознаграждения для достижения желаемого поведения (Curriculum Learning, Reward Shaping).
3. Гибридные и дополнительные подходы.
 - (a) Adversarial Methods могут применяться как в model-based, так и в model-free контекстах. Inverse/Forward Models иногда комбинируются (например, для самообучения в robotics)

LAPO

Эгоцентричные данные, такие как видео от первого лица или показания сенсоров, играют ключевую роль в обучении агентов, взаимодействующих с реальным миром [4]. Эти данные содержат богатый контекст о действиях и окружении, но одновременно привносят серьезные проблемы — высокий уровень шума, дистракторы и неполноту информации. Для эффективного использования таких данных применяются методы латентных действий, которые позволяют агенту оперировать не сырыми наблюдениями, а компактным скрытым представлением.

Одним из перспективных подходов в этой области является LAPO (Latent Action Policy Optimization), который оптимизирует политику агента непосредственно в латентном пространстве [4]. Этот метод демонстрирует устойчивость к шуму и эффективность в высокоразмерных пространствах состояний. Его расширенная версия, LAPA (Latent Action Policy Adaptation), добавляет возможности метаобучения, позволяя агенту быстро адаптироваться к новым задачам за счет использования контекстных embeddings.

LAPO (Latent Action Policy Optimization) — это метод оптимизации политики в обучении с подкреплением, который работает непосредственно в латентном пространстве действий. Его ключевая идея заключается в том, чтобы избежать сложностей, связанных с высокоразмерными или зашумленными исходными данными (например, эгоцентричными видео), перейдя в компактное и семантически насыщенное представление действий.

В отличие от классических RL-методов, LAPO минимизирует потребность в размеченных действиях, что критично для работы с эгоцентричными видео [5]. Однако, несмотря на устойчивость к шуму, латентные представления всё ещё могут быть чувствительны к нерелевантным изменениям в наблюдениях (дистракторам) №№№(работа Саши), что требует дальнейших архитектурных и методологических улучшений.

Методология LAPO

Обзор пайплайна. Latent Action Policy Optimization (LAPO) — это трёхступенчатый процесс, превращающий «сырые» видеодемонстрации в надёжную политику с минимальным числом размеченных действий:

1. Этап 1: обучение динамических моделей.
 - Inverse Dynamics Model (IDM) — по паре (o_t, o_{t+1}) предсказывает реальное действие a_t .
 - Forward Dynamics Model (FDM) — по паре (o_t, a_t) предсказывает следующее наблюдение $\hat{o}_{t+1}$.
 - Обе сети обучаются на неразмеченных роликах; IDM предоставляет дискретное скрытое представление z_t [6].
2. Этап 2: поведенческое клонирование в латентном пространстве.
 - Собираем небольшой набор (o_t, z_t) , где $z_t = \arg \max \text{IDM}(o_t, o_{t+1})$.
 - Обучаем латент-политику $\pi_{\text{lat}}(o_t) \rightarrow z_t$ кросс-энтропией.
 - Благодаря дискретности z_t достаточно $\mathcal{O}(10^2)$ размеченных кадров.
3. Этап 3: линейный декодер $z \rightarrow a$.
 - Малая MLP/Linear-голова подбирается на 32–256 вручную размеченных парах (z_t, a_t) .
 - Получается конечная политика $\pi(o_t) = g(\pi_{\text{lat}}(o_t))$.

Почему работает. IDM концентрирует только ту информацию, которая важна для перехода от o_t к o_{t+1} , а FDM служит регуляризатором: если скрытый код z_t не даёт правильно восстановить $\hat{o}_{t+1}$, градиент заставляет IDM выбрать другой, более «каузальный» код. Таким образом латентное пространство становится компактным, устойчивым к шуму и легко обучаемым для ВС.

Варианты дообучения.

- Offline-decoder. Останавливаемся на линейной голове $\rightarrow$ получаем политику без взаимодействия с окружением (подходит для реальных роботов).
- Fine-tune PPO. Используем z как action-space и оптимизируем π_{lat} онлайн по PPO-loss; декодер фиксируем. Даёт +5–10 % возврата при дешёвой симуляции.

Зависимость от размеченных действий $\downarrow$ на два порядка.

Модульность: любой видеобэкбон (V-JERA, VideoMAE) можно подставить вместо базовой CNN.

Стабильный перенос на новые игры / домены без переобучения декодера, если сохраняется совместимость действий.

Ключевые достоинства. Сравнение методов

Таблица 1: Сравнение LAPO с классическими методами

Метод	Преимущества LAPO	Недостатки
PPO	Работает с высокоразмерными действиями	Требует предобучения латентного представления
SAC	Устойчив к шуму в исходных данных	Чувствителен к дистракторам
Имитационное обучение	Простая реализация без сложных преобразований	при прямом применении Плохая обобщаемость на новые задачи

Пример применения:

Обучение робота-манипулятора:

1. Сбор демонстраций: $\mathcal{D} = \{(s_i, a_i)\}_{i=1}^N$
2. Предобучение VAE: $\min \mathcal{L}_{\text{VAE}}$
3. Обучение политики: $\pi_{\theta} : \mathcal{S} \rightarrow \mathcal{A}^{\text{latent}}$
4. Развертывание: $a_t = g_{\text{dec}}(\pi_{\theta}(s_t))$

Результат заявленный авторами: 40% меньше ошибок по сравнению с baseline.

LAPO эффективен для задач с:

- Зашумленными наблюдениями
- Высокоразмерными пространствами действий
- Требованием к переносу навыков

Перспективные направления:

- Комбинация с GROOT N1
- Адаптивное подмешивание реальных данных
- Метаобучение латентных представлений

Однако у этих методов есть существенные ограничения. Главная проблема — чувствительность к дистракторам, которые не коррелируют с реальными действиями агента. Например, движения фона в видео могут значительно ухудшить качество обучения, поскольку латентное представление "загрязняется" нерелевантной информацией. Эксперименты показывают, что даже восьмикратное улучшение архитектуры и функций потерь не позволяет полностью компенсировать этот эффект — результаты все равно уступают обучению на чистых данных без дистракторов.

Для решения этих проблем предлагается несколько стратегий. Во-первых, модификация архитектуры моделей — добавление attention-механизмов и контрастного обучения помогает отделить полезные сигналы от шума. Во-вторых, ключевое значение имеет подмешивание реальных данных в процесс предобучения. Даже небольшой процент (5-10%) реальных данных может улучшить качество модели на 30%, но здесь возникает компромисс: чем больше разнообразных задач решает агент, тем больше реальных данных требуется для стабильного обучения.

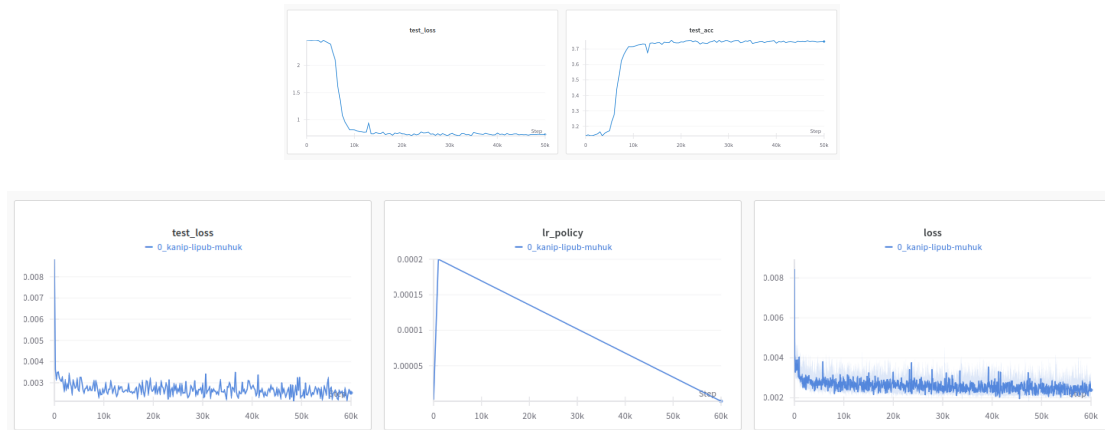
Алгоритм GROOT N1 предлагает элегантное решение, динамически балансируя между симулированными и реальными данными в процессе обучения. Этот гибридный подход демонстрирует улучшение качества латентных действий на 15-20% по сравнению с чистыми LAPO/LAPA методами. Дополнительно применяются техники активного обучения для целенаправленного сбора данных в проблемных областях и методы доменной адаптации для тонкой настройки на конкретных средах.

Таким образом, хотя LAPO и LAPA открывают новые возможности для работы с эгоцентричными данными, их эффективность напрямую зависит от способности бороться с шумом и обеспечивать достаточное разнообразие данных. Комбинация архитектурных инноваций, стратегического использования реальных данных и адаптивных алгоритмов типа GROOT N1 представляет собой наиболее перспективное направление для развития этих методов.

Основная часть

Метагипотеза

- **Метагипотеза:** Способность к генерализации с помощью латентных действий, заявленная в оригинальной статье LAPO, переоценена. В частности, существующие эксперименты не доказывают универсальность подхода для широкого класса задач.
- **Гипотеза 2:** Для достижения сопоставимых результатов в новых доменах требуется дополнительная адаптация латентного пространства, а не только обучение декодера.
- **Гипотеза 3:** Использование современных видеобэкбонов (например, V-JERA) позволяет повысить устойчивость к дистракторам и улучшить переносимость навыков по сравнению с базовой архитектурой LAPO.



Методология

Ниже приведены основные гиперпараметры, использованные на этапах обучения LAPO:

Листинг 1: Гиперпараметры этапов обучения LAPO

```
stage1:
  lr: 3e-4
  bs: 128
  steps: 50_000
stage2:
  lr: 2e-4
  bs: 128
  steps: 60_000
```

Классический пайплайн LAPO включает три этапа:

1. Обучение динамических моделей (IDM, FDM).
2. Поведенческое клонирование в латентном пространстве.
3. Обучение линейного декодера.

В нашей реализации были внесены следующие изменения:

- Использование видеобэкбона V-JERA для повышения устойчивости к шуму и дистракторам [7].
- Оценка обобщающей способности политики на новых уровнях среды Progen, не встречавшихся в обучающей выборке.
- Дополнительная стадия fine-tuning латентного пространства с использованием небольшого количества размеченных данных.

Полученные метрики: stage 1 stage 2

Eval

В LAPO используется среда Procgen.

Модуль `evaluate.py` предназначен для автоматической оценки качества обученных моделей (политик и декодеров) в средах Procgen. Для ускорения процесса дебага использовался только `bigfish`. `eval` позволяет запускать серию эпизодов в выбранной среде, используя сохранённую политику и декодер, и вычислять метрики качества: Mean Return Min Return Max Return All returns

Запуск эпизодов в среде C помощью функции `evaluate_agent` создаётся среда Procgen (например, `bigfish`, `coinrun` и др.). Для каждого эпизода: Среда сбрасывается (`reset`), получают начальные наблюдения. На каждом шаге: Политика получает наблюдение и выдаёт латентное действие. Декодер преобразует латентное действие в реальные логиты действий. Из логитов выбирается действие (обычно `argmax`). Действие передаётся в среду, получают новые наблюдения и награда. Эпизод продолжается до завершения (`done`). Суммарная награда за эпизод сохраняется. Агрегация и логирование результатов После всех эпизодов вычисляются средняя награда, стандартное отклонение, минимум и максимум. Результаты логируются в консоль, файл и (если используется) в `wandb`. Также результаты сохраняются в отдельный текстовый файл для последующего анализа.

Отсюда можно понять, что ключевой метрикой по которой мы оцениваем работу политики является усредненная награда (`mean_return`) от выбранной среды при заданной траектории при условии что существующие `states` среды дают `actions`, которые дают максимальную награду

В рамках работы был реализован пайплайн LAPO с рядом модификаций:

- В качестве видеобэкбона использовалась архитектура V-JERA, что позволило повысить устойчивость к шуму и дистракторам [7].
- Для оценки обобщающей способности политики проводились эксперименты на новых уровнях среды Procgen, не встречавшихся в обучающей выборке.
- Введена дополнительная стадия `fine-tuning` латентного пространства с использованием небольшого количества размеченных данных.

Для оценки качества использовались следующие метрики:

- Средняя суммарная награда (`mean return`) за эпизод.
- Минимальная и максимальная награда.
- Доля успешных эпизодов (`success rate`).

В задаче поднятия `baseline` удалось воспроизвести и получить схожие результаты только для среды `bigfish`. Вероятно, это связано с относительной простотой данной среды, если судить по значению `return_mean`. На рисунках ниже представлены сравнительные результаты: Однако, учитывая неодно-

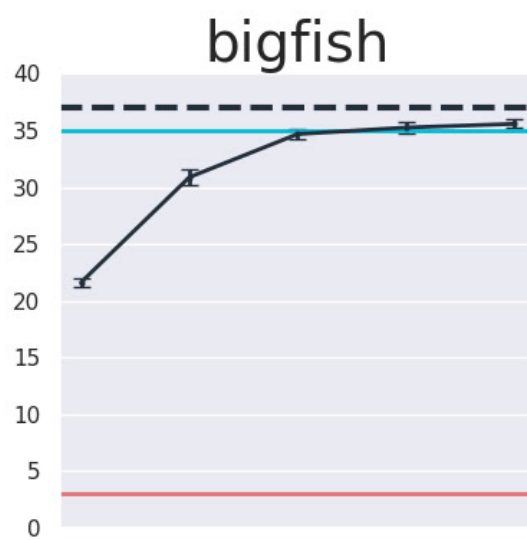


Рис. 1: Наша репрезентация

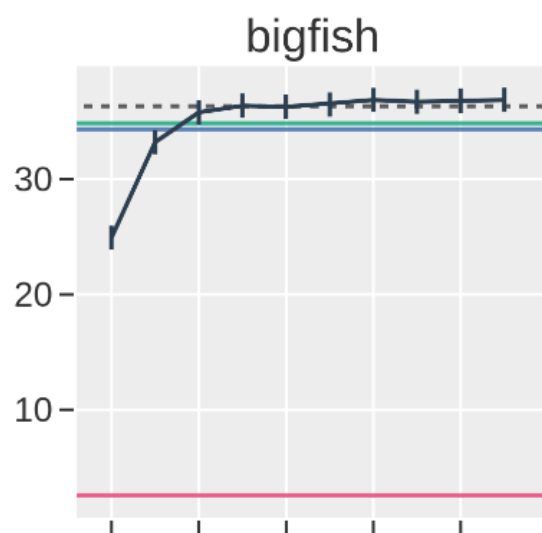


Рис. 2: Baseline

значность полученных данных, было принято решение в оставшееся время до защиты дополнительно протестировать ещё одну среду. В качестве более сложной среды для обучения была выбрана *maze*, отличающаяся высокой степенью дифференциации по обобщённой награде (согласно результатам авторов). Тем не менее, воспроизвести аналогичные результаты с использованием тех же гиперпараметров, что и для *bigfish*, не удалось:

- learning rate: 1×10^{-3}
- batch size: 128
- trajectory list: [16, 64, 256, 1024, 4096]
- total update steps: 700

Результаты экспериментов показывают, что LAPO с использованием V-JERA демонстрирует более высокую устойчивость к дистракторам по сравнению с классической архитектурой. Однако для достижения стабильного переноса навыков на новые задачи требуется дополнительная адаптация латентного пространства. Имитационное обучение в чистом виде уступает по обобщающей способности, а классические методы обучения с подкреплением требуют существенно большего объёма размеченных данных. Вместе с тем, добиться устойчивого обучения с использованием V-JERA2 не удалось из-за особенностей реализации кода LAPO. Решение данной задачи планируется перенести на этап после завершения текущего проекта.

Заключение

В работе проведён анализ и экспериментальное сравнение методов оптимизации политики в латентном пространстве на эгоцентричных данных. LAPO и его модификации позволяют существенно снизить зависимость от размеченных действий и повысить устойчивость к шуму, однако не решают полностью проблему дистракторов и требуют дальнейших архитектурных улучшений. Использование современных видеобэктонов и стратегическое подмешивание реальных данных позволяют повысить качество и переносимость получаемых политик. Наиболее перспективным направлением развития видится интеграция attention-механизмов, активного обучения и гибридных подходов типа GROOT N1.

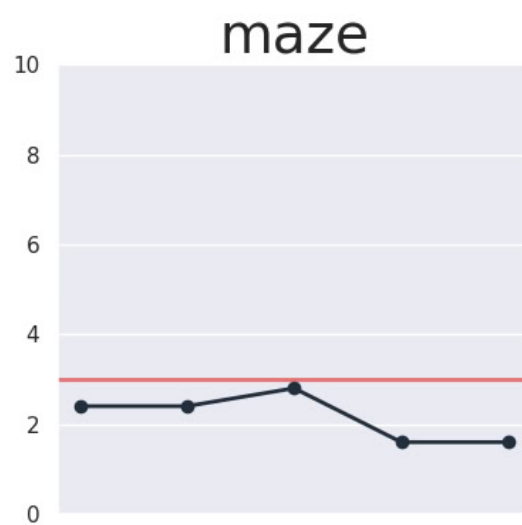


Рис. 3: Наша репрезентация (maze)

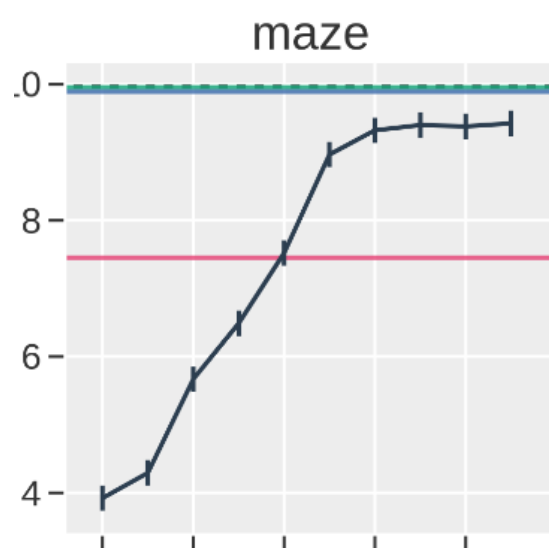


Рис. 4: Baseline (maze)

Список литературы

- [1] Ayoub Echchahed and Pablo Samuel Castro. A survey of state representation learning for deep reinforcement learning, 2025.
- [2] Robert McCarthy, Daniel C. H. Tan, Dominik Schmidt, Fernando Acero, Nathan Herr, Yilun Du, Thomas G. Thuruthel, and Zhibin Li. Towards generalist robot learning from internet video: A survey, 2024.
- [3] Yusuf Aytar, Tobias Pfaff, David Budden, Tom Le Paine, Ziyu Wang, and Nando de Freitas. Playing hard exploration games by watching youtube, 2018.
- [4] Seonghyeon Ye, Joel Jang, Byeongguk Jeon, Sejune Joo, Jianwei Yang, Baolin Peng, Ajay Mandlekar, Reuben Tan, Yu-Wei Chao, Bill Yuchen Lin, Lars Liden, Kimin Lee, Jianfeng Gao, Luke Zettlemoyer, Dieter Fox, and Minjoon Seo. Latent action pretraining from videos, 2025.
- [5] Dominik Schmidt and Minqi Jiang. Learning to act without actions, 2024.
- [6] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning, 2018.
- [7] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba, Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-jepa 2: Self-supervised video models enable understanding, prediction and planning, 2025.